SSL Manager — Administrator Guide

SSL Manager — System Requirements

Table of Contents

---

1. Operating System

Ubuntu (recommended)

Requirement	Value
Supported OS	Ubuntu 24.04 LTS (Noble Numbat) — recommended
Also supported	Ubuntu 22.04 LTS (Jammy Jellyfish), Ubuntu 20.04 LTS (Focal Fossa)
Architecture	x86_64 (amd64), arm64 (aarch64)
Install type	Server (minimal) or standard server image

Ubuntu 24.04 LTS is the recommended target. The installer (`install.sh`) detects the OS and enforces Ubuntu 20.04 or later.

RHEL-family

Requirement	Value
Supported OS	RHEL 9, Rocky Linux 9, AlmaLinux 9, CentOS Stream 9
Also supported	RHEL 8, Rocky Linux 8, AlmaLinux 8
Architecture	x86_64 (amd64), arm64 (aarch64)
Install type	Minimal install
Installer	`install-rhel.sh` (separate from the Ubuntu `install.sh`)

RHEL-family deployments require SELinux configuration, which `install-rhel.sh` handles automatically. See the RHEL section of INSTALL.md for full instructions.

---

2. Minimum Hardware

The following sizing targets installations with up to **1,000 certificate records** and **2–3 concurrent users** accessing via SSH tunnel.

Resource	Minimum	Recommended
CPU	1 vCPU	2 vCPU
RAM	1 GB	2 GB
Disk	10 GB SSD	20 GB SSD
Network	Any	Any

2.1 Sizing Rationale

The table below shows the expected steady-state memory footprint of all running components:

Component	Memory	Notes
Ubuntu 24.04 minimal OS	200–300 MB	systemd, journald, base services
nginx (2 workers)	10–20 MB	Loopback reverse proxy + static file serving
gunicorn (2 workers)	100–160 MB	Each Python worker loads Flask, SQLAlchemy, and the cryptography library (~50–80 MB each)

Component	Memory	Notes
SQLite	5–10 MB	Page cache; under 1,000 records the database remains small
fail2ban	30–50 MB	Python-based log watcher; resident in memory at all times
sshd (2–3 active tunnels)	15–30 MB	~5–10 MB per active SSH tunnel session
Total steady-state	**~360–570 MB**	

1 GB RAM runs all components comfortably with 40–60% headroom under normal load.

CPU is nearly idle between requests. The two operations that briefly spike CPU are:

- **RSA key generation** — RSA-2048 takes 50–200 ms per key; RSA-4096 takes 500 ms–2 s on a single vCPU. Each generation occupies one gunicorn worker for its duration.
- **Certificate format export** (PKCS#12, JKS, P7B) — subprocess calls to `openssl` and `keytool`, each completing in under 1 second.

With 2 gunicorn workers, one worker can generate a key while another concurrently serves a page request.

Disk usage for 1,000 certificates:

Item	Size
Ubuntu OS + installed packages	3–5 GB
Python virtual environment + app files	150–200 MB
SQLite database (1,000 certs × ~8 KB PEM data)	20–50 MB
7-day backup retention (PEM text compresses ~75% with gzip)	20–50 MB
nginx + gunicorn logs (1 year, light usage)	100–300 MB
Total	**~4–6 GB**

A **10 GB** disk is sufficient; **20 GB** provides comfortable headroom for log accumulation and OS updates.

2.2 When to Upsize

Scenario	Recommendation
fail2ban configured with aggressive regex scanning against large nginx log files	2 GB RAM
Server runs other services alongside SSL Manager	2 GB RAM
Users frequently generate RSA-4096 keys concurrently (3 simultaneous users can briefly queue behind 2 workers)	2 vCPU
Backup retention extended significantly beyond 7 days	Larger disk
Certificate records exceed ~10,000	Review disk and RAM; SQLite remains performant well past this point

Representative instance tiers matching the minimum and recommended specifications:

Provider	Tier	Spec	Est. Cost	Notes
Hetzner	CX11	2 vCPU / 2 GB / 20 GB SSD	~€4/month	Best value; storage included
DigitalOcean	Basic Droplet	1 vCPU / 1 GB / 25 GB SSD	~$6/month	Storage included
Linode/Akamai	Nanode	1 vCPU / 1 GB / 25 GB SSD	~$5/month	Storage included
Vultr	Cloud Compute	1 vCPU / 1 GB / 25 GB SSD	~$6/month	Storage included
AWS	t4g.micro (minimum)	2 vCPU / 1 GB	~$6/month	Add ~$1.60/ month for 20 GB gp3 EBS; ARM Graviton2; Ubuntu 24.04 supported
AWS	t4g.small (recommended)	2 vCPU / 2 GB	~$12/month	Add ~$1.60/ month for 20 GB gp3 EBS; ARM Graviton2; eligible for Savings Plans
Azure	B1s (minimum)	1 vCPU / 1 GB	~$8/month	4 GB managed OS disk included; Standard HDD additional
Azure		1 vCPU / 2 GB	~$15/month	

Provider	Tier	Spec	Est. Cost	Notes
	B1ms (recommended)			4 GB managed OS disk included; add Standard SSD for data

> **AWS pricing notes:** EC2 compute and EBS storage are billed separately. A 20 GB gp3 EBS volume adds approximately $1.60/month. New AWS accounts receive 750 hours/month of t2.micro or t3.micro (x86) On-Demand usage free for 12 months under the Free Tier. Prices shown are On-Demand Linux rates for us-east-1; Reserved Instances (1-year, no upfront) reduce compute cost by approximately 30–40%.
>
> **Azure pricing notes:** Azure VM pricing includes a small managed OS disk but application data and backups should use a separate managed disk. Prices shown are Pay-As-You-Go Linux rates for East US. Azure Reserved VM Instances (1-year) reduce cost by approximately 30–40%. The B-series (burstable) is appropriate for this workload as CPU usage is low between certificate operations.
>
> **All prices are estimates** based on public list pricing at time of writing and will vary by region, commitment term, and provider promotions. Verify current pricing on each provider's pricing page before provisioning.

Hetzner CX11 offers the best value for this workload — 2 vCPU and 2 GB RAM with storage included at the lowest monthly cost. AWS and Azure become more competitive when an organisation already has existing cloud credits, reserved capacity, or consolidated billing agreements.

---

3. System Packages

3.1 Installed by the Installer

Ubuntu (`install.sh`)

The packages below are installed automatically by `install.sh`:

Package	Purpose
`nginx`	Reverse proxy; listens on `127.0.0.1` only; serves static files directly and forwards dynamic requests to gunicorn via Unix socket
`python3-pip`	

Package	Purpose
	pip package manager used to install the Python virtual environment dependencies
`python3-venv`	Creates the isolated Python virtual environment at `/opt/ssl-manager/venv`
`python3-dev`	C header files required to compile the `cryptography` package's native extensions
`gcc`	C compiler required to build the `cryptography` package during `pip install`

> **`python3`** and **`openssl`** are present in a default Ubuntu 24.04 LTS server install; the installer includes them in the `apt-get install` call as an explicit dependency declaration.
>
> **`sqlite3`** (the CLI tool) is required by `backup.sh` for WAL checkpointing and database integrity checks, but is **not installed automatically** by `install.sh`. On Ubuntu 24.04 minimal server it may not be present. If backups fail with `sqlite3 not found`, install it manually:
>
> ```
> sudo apt-get install -y sqlite3
> ```

RHEL-family (`install-rhel.sh`)

The packages below are installed automatically by `install-rhel.sh`:

Package	Purpose
`nginx`	Reverse proxy (same role as Ubuntu)
`python3-pip`	pip package manager
`python3-devel`	C header files for native extension builds (RHEL equivalent of `python3-dev`)
`gcc`	C compiler
`policycoreutils-python-utils`	Provides `semanage`, used to configure SELinux file contexts and port permissions
`epel-release`	Extra Packages for Enterprise Linux — installed automatically on Rocky/AlmaLinux/CentOS Stream; skipped on registered RHEL (use subscription-manager instead)

> **`python3`** and **`openssl`** are present in RHEL 9 / Rocky 9 minimal installs.

3.2 Recommended Hardening Packages

The following packages are **not installed by either installer** because they affect system-wide services and require manual review before enabling.

Package	Ubuntu	RHEL-family	Purpose
`ufw`	Included, inactive	Not available (use `firewalld`)	Host firewall; restrict inbound traffic to SSH only
`firewalld`	Not default	Included, active	RHEL host firewall; restrict inbound traffic to SSH only
`fail2ban`	Not included	Not included	Monitors nginx and sshd logs; bans IPs that repeatedly fail authentication
`unattended-upgrades`	Included, inactive	Not available (use `dnf-automatic`)	Automatically installs security updates
`dnf-automatic`	Not available	Not included	RHEL equivalent of unattended-upgrades

Optional — JKS export format:

Package	Ubuntu	RHEL-family	Purpose
`default-jdk-headless`	Not included	—	Provides `keytool` for Java KeyStore (`.jks`) export
`java-17-openjdk-headless`	—	Not included	RHEL equivalent; provides `keytool`

Install on Ubuntu:

```
sudo apt-get install -y default-jdk-headless
```

Install on RHEL-family:

```
sudo dnf install -y java-17-openjdk-headless
```

If `keytool` is not installed, the JKS download button returns an error. All other export formats continue to work.

4. Python Packages

All Python dependencies are installed into an isolated virtual environment at `/opt/ssl-manager/venv` and are **not installed system-wide**.

Package	Version	Purpose
`Flask`	3.1.3	Web framework — routing, templating, session management
`Flask-SQLAlchemy`	3.1.1	SQLAlchemy ORM integration for Flask; manages the SQLite connection pool
`Flask-Login`	0.6.3	User session management, login/logout, `current_user` context, `@login_required` decorator
`cryptography`	46.0.7	RSA key generation, CSR construction, certificate parsing, PKCS#12 export; uses OpenSSL bindings
`pyjks`	20.0.0	Java KeyStore (JKS) file generation for the `.jks` download format
`gunicorn`	23.0.0	Production WSGI server; runs the Flask app as multiple worker processes behind the nginx Unix socket

The `pip install` step runs during `install.sh` / `install-rhel.sh` and again during `--upgrade`.

---

5. Network Requirements

SSL Manager is designed to be **unreachable from the network** by default. No inbound port other than SSH needs to be open.

Interface	Port	Protocol	Direction	Purpose
All interfaces	22	TCP	Inbound	SSH — required for tunnel access and server administration

Interface	Port	Protocol	Direction	Purpose
`127.0.0.1`	5001 (configurable)	TCP	Loopback only	nginx listener; not exposed to the network
`/run/ssl-manager/ssl-manager.sock`	—	Unix socket	Internal	nginx → gunicorn communication
Any	25, 465, or 587	TCP	**Outbound**	SMTP — only required if email (password reset) is configured

Remote access is provided exclusively via SSH port forwarding:

```
ssh -L 5001:127.0.0.1:5001 user@your-server
```

No HTTP/HTTPS ports are opened to the network. A firewall rule allowing only port 22 inbound is sufficient for all functionality except outbound email.

Outbound SMTP is only needed if the password reset feature is configured. The specific port depends on your mail provider:

Auth method	Typical port
STARTTLS	587
SSL/TLS	465
Plain SMTP (internal relay, not recommended)	25

---

6. File System Layout

All paths created by `install.sh` / `install-rhel.sh`:

Path	Owner	Mode	Purpose
`/opt/ssl-manager/`	`root:ssl-manager`	`750`	Application code, templates, static files, Python venv
`/opt/ssl-manager/venv/`	`root:ssl-manager`	`750`	Isolated Python virtual environment
`/opt/ssl-manager/backup.sh`	`root:ssl-manager`	`750`	Database backup script
		`700`	

Path	Owner	Mode	Purpose
`/var/lib/ssl-manager/`	`ssl-manager:ssl-manager`		SQLite database — accessible only by the service user
`/var/lib/ssl-manager/ssl_manager.db`	`ssl-manager:ssl-manager`	`600`	Live SQLite database (WAL mode)
`/var/log/ssl-manager/`	`ssl-manager:ssl-manager`	`750`	gunicorn access and error logs
`/var/backups/ssl-manager/`	`root:root`	`700`	Compressed database backups (7-day retention by default)
`/etc/ssl-manager/`	`root:ssl-manager`	`750`	Application configuration directory
`/etc/ssl-manager/env`	`root:ssl-manager`	`640`	Environment file — `SECRET_KEY`, `DATABASE_URL`
`/run/ssl-manager/ssl-manager.sock`	`ssl-manager:ssl-manager`	`660`	Unix socket between nginx and gunicorn (recreated at each service start)
`/etc/systemd/system/ssl-manager.service`	`root`	`644`	gunicorn systemd service unit
`/etc/systemd/system/ssl-manager-backup.service`	`root`	`644`	Backup job systemd unit (Type=oneshot)
`/etc/systemd/system/ssl-manager-backup.timer`	`root`	`644`	Backup timer unit (daily at 02:00, Persistent=true)

Ubuntu-specific paths:

Path	Owner	Mode	Purpose
`/etc/nginx/sites-available/ssl-manager`	`root`	`644`	nginx reverse proxy configuration
`/etc/nginx/conf.d/ssl-manager-ratelimit.conf`	`root`	`644`	nginx rate-limit zone definition (10 req/s per IP)

RHEL-family-specific paths:

Path	Owner	Mode	Purpose
`/etc/nginx/conf.d/ssl-manager.conf`	`root`	`644`	nginx reverse proxy configuration (RHEL uses conf.d; no sites-available)
`/etc/nginx/conf.d/ssl-manager-ratelimit.conf`	`root`	`644`	nginx rate-limit zone definition

SSL Manager — Installation Guide

Deployment Options

Method	Best for
Bare Metal / Ubuntu Server	Physical hardware, on-premises VMs, self-managed VPS
RHEL / Rocky Linux / AlmaLinux	Red Hat-based enterprise or on-premises environments
AWS (EC2)	Existing AWS accounts, consolidated billing, Graviton cost savings
Azure (VM)	Existing Azure accounts, enterprise agreements, Azure credits
Docker (local development)	Development, testing, and feature validation — not for production

Cloud deployments (AWS and Azure) require Terraform and the relevant cloud CLI. See Cloud Deployment Prerequisites for installation instructions covering macOS, Linux, and Windows — including SSH key generation for all platforms and server-side user management.

All production methods share the same runtime stack: nginx → gunicorn → Flask → SQLite, with access via SSH tunnel only. See REQUIREMENTS.md for hardware sizing guidance.

Bare Metal / Ubuntu Server

Use this method for physical hardware, on-premises virtual machines, or any VPS where you provision the OS yourself.

Requirements

- Ubuntu 24.04 LTS (recommended), 22.04 LTS, or 20.04 LTS
- Root or `sudo` access
- Outbound internet access (for `apt-get` and `pip`)
- Minimum: 1 vCPU, 1 GB RAM, 10 GB disk
- Recommended: 1–2 vCPU, 2 GB RAM, 20 GB disk

1. Get the code

```
git clone https://github.com/your-org/ssl-manager.git
cd ssl-manager
```

Or transfer the files to the server:

```
scp -r ssl-manager/ user@your-server:~/ssl-manager
```

2. Run the installer

```
sudo bash install.sh
```

The installer prompts for three values:

Prompt	Default	Notes
nginx listen port	`5001`	nginx binds to `127.0.0.1:<port>` — loopback only, not reachable from the network
Gunicorn worker processes	`2`	Increase for higher concurrent load; 2 is sufficient for 2–3 users
Secret key	auto-generated	256-bit random hex used to sign Flask sessions; leave blank to auto-generate

The installer:

1. Installs system packages (`nginx`, `python3-pip`, `python3-venv`, `gcc`, `python3-dev`)
2. Creates the `ssl-manager` service account (no login shell, no home directory)
3. Creates all directories with enforced ownership and permissions
4. Copies application files to `/opt/ssl-manager/`
5. Creates a Python virtual environment and installs all dependencies
6. Writes the secret key and database URL to `/etc/ssl-manager/env` (on a re-run, an existing `SECRET_KEY` is preserved — never regenerated — because it encrypts stored SMTP/OAuth secrets)

7. Installs and starts the `ssl-manager` systemd service
8. Configures nginx as a reverse proxy with rate limiting
9. Installs and enables the `ssl-manager-backup.timer` for daily database backups

> **sqlite3:** The backup script requires the `sqlite3` CLI. Ubuntu 24.04 minimal server may not include it. If the first scheduled backup fails, install it with `sudo apt-get install -y sqlite3`.

> **Re-running the installer:** Running `sudo bash install.sh` again on a host that already has SSL Manager auto-detects the existing installation and switches to **upgrade mode** — it skips the prompts and preserves your configuration, secret key, and database. See Upgrading. Use `--reinstall` to force the interactive installer instead.

3. Verify the service is running

```
sudo systemctl status ssl-manager
sudo systemctl status ssl-manager-backup.timer
```

4. Connect via SSH tunnel

From your local machine:

```
ssh -L 5001:127.0.0.1:5001 user@your-server
```

Open `http://localhost:5001` in your browser. Complete the first-run setup to create the superadmin account.

For a persistent tunnel, add to `~/.ssh/config`:

```
Host ssl-manager
    HostName your-server
    User your-user
    LocalForward 5001 127.0.0.1:5001
    ServerAliveInterval 60
```

Then `ssh ssl-manager` is all that is needed.

5. Recommended post-install hardening

The following steps are not automated because they affect system-wide services. Review each before applying.

Firewall (UFW):

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow ssh
sudo ufw enable
sudo ufw status verbose
```

fail2ban:

```
sudo apt-get install -y fail2ban
sudo systemctl enable --now fail2ban
```

Create `/etc/fail2ban/jail.d/ssl-manager.conf`:

```
[sshd]
enabled  = true
port     = ssh
maxretry = 5
bantime  = 1h
findtime = 10m

[nginx-limit-req]
enabled  = true
port     = http,https
logpath  = /var/log/nginx/ssl-manager-error.log
maxretry = 10
bantime  = 1h
```

```
sudo systemctl restart fail2ban
```

Unattended security updates:

```
sudo dpkg-reconfigure --priority=low unattended-upgrades
```

Service management

```
# Status and logs
sudo systemctl status ssl-manager
sudo journalctl -u ssl-manager -f
sudo tail -f /var/log/ssl-manager/error.log

# Restart after config change
sudo nano /etc/ssl-manager/env
sudo systemctl restart ssl-manager

# Backup management
sudo systemctl status ssl-manager-backup.timer
sudo systemctl list-timers ssl-manager-backup.timer
sudo bash /opt/ssl-manager/backup.sh   # run a backup immediately
```

Upgrading

After pulling or transferring updated code:

```
sudo bash install.sh --upgrade
```

Re-running the installer with no flag does the same thing — it auto-detects the existing install and upgrades.

The upgrade backs up the database, then refreshes the application files, Python dependencies, systemd unit, and nginx config from the new code. Your `SECRET_KEY`, database, and existing port/worker settings are **preserved** (the port and worker count are re-derived from the current nginx/systemd config, so there are no prompts). Schema migrations run automatically on the next service start — no manual SQL steps are required. See Upgrading from Previous Versions for version-specific notes.

Uninstalling

```
sudo bash install.sh --uninstall
```

Removes the service, application files, nginx config, and optionally the database and service user. The database at `/var/lib/ssl-manager/` is not deleted unless explicitly confirmed.

RHEL / Rocky Linux / AlmaLinux

Use this method for RHEL 8/9, Rocky Linux 8/9, AlmaLinux 8/9, or CentOS Stream 9. The RHEL installer (`install-rhel.sh`) handles SELinux configuration and `dnf`-based package management automatically.

Requirements

- RHEL 9 / Rocky Linux 9 / AlmaLinux 9 (recommended), or 8-series equivalents
- Root or `sudo` access
- Outbound internet access (for `dnf` and `pip`)
- Minimum: 1 vCPU, 1 GB RAM, 10 GB disk
- Recommended: 1–2 vCPU, 2 GB RAM, 20 GB disk
- **Registered RHEL only:** An active Red Hat subscription with the AppStream and BaseOS repositories enabled

1. Get the code

```
git clone https://github.com/your-org/ssl-manager.git
cd ssl-manager
```

Or transfer the files to the server:

```
scp -r ssl-manager/ user@your-server:~/ssl-manager
```

2. Run the installer

```
sudo bash install-rhel.sh
```

The installer prompts for the same three values as the Ubuntu installer (port, worker count, secret key).

The installer additionally:

1. Enables EPEL (on Rocky/AlmaLinux/CentOS Stream; skipped on registered RHEL)

2. Installs `policycoreutils-python-utils` for SELinux management
3. Configures SELinux to allow nginx to communicate with the gunicorn Unix socket:
    - Sets `httpd_var_run_t` file context on the socket directory
    - Enables `httpd_can_network_connect` boolean
    - Adds the chosen port to the `http_port_t` SELinux port type
4. Writes the nginx config to `/etc/nginx/conf.d/` (RHEL uses `conf.d`; there is no `sites-available` / `sites-enabled`)
5. Uses `nginx` as the nginx user (not `www-data` as on Ubuntu)

> **SELinux note:** If you later change the nginx port after installation, re-run the SELinux port permission manually:
>
> ```
> sudo semanage port -a -t http_port_t -p tcp <new-port>
> sudo systemctl restart nginx
> ```

3. Verify the service is running

```
sudo systemctl status ssl-manager
sudo systemctl status ssl-manager-backup.timer
```

4. Connect via SSH tunnel

Identical to the Ubuntu procedure — from your local machine:

```
ssh -L 5001:127.0.0.1:5001 user@your-server
```

Open `http://localhost:5001` and complete the first-run setup.

5. Recommended post-install hardening

Firewall (firewalld):

```
# Allow SSH only — all other inbound ports are blocked by default on RHEL
sudo firewall-cmd --permanent --add-service=ssh
sudo firewall-cmd --permanent --remove-service=cockpit   # optional
sudo firewall-cmd --reload
sudo firewall-cmd --list-all
```

fail2ban:

```
sudo dnf install -y fail2ban
sudo systemctl enable --now fail2ban
```

Create `/etc/fail2ban/jail.d/ssl-manager.conf` with the same content as the Ubuntu example above, then:

```
sudo systemctl restart fail2ban
```

Automatic security updates (dnf-automatic):

```
sudo dnf install -y dnf-automatic
sudo sed -i 's/^apply_updates = .*/apply_updates = yes/' /etc/dnf/automatic.conf
sudo systemctl enable --now dnf-automatic.timer
```

Service management

Identical commands to Ubuntu — `systemctl status ssl-manager`, `journalctl -u ssl-manager`, etc.

Upgrading

```
sudo bash install-rhel.sh --upgrade
```

As on Ubuntu, re-running `install-rhel.sh` with no flag auto-detects the existing install and upgrades, preserving your `SECRET_KEY`, database, and port/worker settings (and re-applying the SELinux and nginx configuration). Use `--reinstall` to force the interactive installer. See Upgrading from Previous Versions for version-specific notes.

Uninstalling

```
sudo bash install-rhel.sh --uninstall
```

Configure Email (Optional)

SSL Manager can send password reset emails via SMTP. This step is optional — the application functions fully without email; only the password reset flow requires it.

Supported authentication methods

Method	When to use
Standard SMTP	Any mail provider that supports SMTP with username/password — Gmail (app password), SendGrid, Amazon SES, Mailgun, your own server
Microsoft 365 OAuth	Microsoft 365 / Exchange Online accounts where modern authentication is required
Google OAuth	Google Workspace accounts using OAuth 2.0

Configure via the web UI

1. Log in as a **superadmin** user.
2. Navigate to **Settings** → **Email** (`/settings/smtp`).
3. Fill in the fields for your provider:

Standard SMTP fields:

Field	Example
SMTP Host	`smtp.sendgrid.net`
Port	`587` (STARTTLS) or `465` (SSL)
Username	Your SMTP login or API key name
Password	Your SMTP password or API key
From address	`noreply@yourdomain.com`
From name	`SSL Manager`
Use TLS / Use SSL	Match your provider's requirement

Microsoft 365 OAuth:

1. Register an application in Azure Active Directory.
2. Add the redirect URI: `http://localhost:5001/settings/smtp/oauth/microsoft/callback`

3. Grant the `Mail.Send` permission (delegated).
4. In the SSL Manager SMTP settings, select **Microsoft 365 OAuth**, enter the **Client ID**, **Client Secret**, and **Tenant ID**, then click **Connect** to complete the OAuth flow.

Google OAuth:

1. Create an OAuth 2.0 Client ID in Google Cloud Console.
2. Add the redirect URI: `http://localhost:5001/settings/smtp/oauth/google/callback`
3. Enable the **Gmail API** in your project.
4. In the SSL Manager SMTP settings, select **Google OAuth**, enter the **Client ID** and **Client Secret**, then click **Connect**.

Test email delivery

After saving your settings, click **Send Test Email** on the SMTP settings page. A test message will be sent to your own account (the logged-in superadmin's email address). Check the audit log if the test fails — delivery errors are recorded there.

Network requirements

Ensure outbound TCP is permitted on the port your provider uses:

```
# Ubuntu (UFW)
sudo ufw allow out 587/tcp   # STARTTLS
sudo ufw allow out 465/tcp   # SSL (if used instead)

# RHEL (firewalld)
sudo firewall-cmd --permanent --add-port=587/tcp
sudo firewall-cmd --reload
```

Cloud Deployment Prerequisites

The AWS and Azure deployments both use Terraform. Install the required tools on your local machine before proceeding.

Terraform

macOS

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
terraform version   # confirm install
```

Linux (Ubuntu / Debian)

```
sudo apt-get install -y gnupg software-properties-common
wget -O- https://apt.releases.hashicorp.com/gpg \
  | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
  https://apt.releases.hashicorp.com $(lsb_release -cs) main" \
  | sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt-get update && sudo apt-get install -y terraform
terraform version   # confirm install
```

Linux (RHEL / Fedora / Amazon Linux)

```
sudo yum install -y yum-utils
sudo yum-config-manager --add-repo https://rpm.releases.hashicorp.com/RHEL/hashicorp.repo
sudo yum install -y terraform
terraform version   # confirm install
```

Windows

Option A — winget (Windows 10/11, recommended):

```
winget install HashiCorp.Terraform
terraform version   # confirm install
```

Option B — Chocolatey:

```
choco install terraform
terraform version   # confirm install
```

Option C — Manual: download the `.zip` from developer.hashicorp.com/terraform/downloads, extract `terraform.exe`, and add its folder to your `PATH`.

AWS CLI (required for AWS deployments only)

macOS

```
brew install awscli
aws --version   # confirm install
```

Linux

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip awscliv2.zip
sudo ./aws/install
aws --version   # confirm install
```

For ARM (Graviton) systems replace `x86_64` with `aarch64` in the URL.

Windows

Option A — winget:

```
winget install Amazon.AWSCLI
aws --version   # confirm install
```

Option B — MSI installer: download and run AWSCLIV2.msi from AWS, then confirm with `aws --version` in a new terminal.

Azure CLI (required for Azure deployments only)

macOS

```
brew install azure-cli
az version   # confirm install
```

Linux (Ubuntu / Debian)

```
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
az version   # confirm install
```

Linux (RHEL / Fedora / Amazon Linux)

```
sudo rpm --import https://packages.microsoft.com/keys/microsoft.asc
sudo dnf install -y azure-cli
az version   # confirm install
```

Windows

Option A — winget:

```
winget install Microsoft.AzureCLI
az version   # confirm install
```

Option B — MSI installer: download and run the installer from aka.ms/installazurecliwindows, then confirm with `az version` in a new terminal.

SSH key pair

Every user who needs to connect to the server requires an SSH key pair. Generate one on the machine you will connect from, then provide the public key to the server administrator (or paste it into `terraform.tfvars` for Terraform deployments).

macOS

OpenSSH is included with macOS. Open **Terminal** and check whether a key already exists:

```
ls -la ~/.ssh/id_ed25519 ~/.ssh/id_ed25519.pub 2>/dev/null
```

If a key already exists and you are comfortable reusing it for SSL Manager, skip to the “View / copy” step below.

If no key exists, or you prefer a dedicated key for this server, generate one:

```
# Default name — use if you have no existing key
ssh-keygen -t ed25519 -C "your-name-ssl-manager"

# Named key — use if you want to keep this separate from existing keys
ssh-keygen -t ed25519 -C "your-name-ssl-manager" -f ~/.ssh/id_ed25519_ssl_manager
```

Set a passphrase when prompted — this protects the private key if your machine is compromised.

```
# View the public key (share this — never the private key)
cat ~/.ssh/id_ed25519.pub           # default key
cat ~/.ssh/id_ed25519_ssl_manager.pub  # named key

# Copy the public key to the clipboard
pbcopy < ~/.ssh/id_ed25519.pub
pbcopy < ~/.ssh/id_ed25519_ssl_manager.pub  # named key
```

Keys are stored in `~/.ssh/`:

File	Description
`~/.ssh/id_ed25519`	Private key — keep this secret, never share it
`~/.ssh/id_ed25519.pub`	Public key — safe to share; goes on the server

Connecting with a named key — use the `-i` flag to specify which private key to use:

```
ssh -i ~/.ssh/id_ed25519_ssl_manager user@<server-ip>

# With tunnel
ssh -i ~/.ssh/id_ed25519_ssl_manager -L 5001:127.0.0.1:5001 user@<server-ip>
```

To avoid typing `-i` every time, add an entry to `~/.ssh/config`:

```
Host ssl-manager
    HostName <server-ip>
    User <username>
    IdentityFile ~/.ssh/id_ed25519_ssl_manager
    LocalForward 5001 127.0.0.1:5001
    ServerAliveInterval 60
```

Then connect with just `ssh ssl-manager`.

Linux

OpenSSH is included on most distributions. If it is missing:

```
sudo apt-get install -y openssh-client   # Ubuntu / Debian
sudo dnf install -y openssh              # Fedora / RHEL
```

Check whether a key already exists:

```
ls -la ~/.ssh/id_ed25519 ~/.ssh/id_ed25519.pub 2>/dev/null
```

If no key exists, or you want a dedicated key:

```
# Default name
ssh-keygen -t ed25519 -C "your-name-ssl-manager"

# Named key — keeps this separate from existing keys
ssh-keygen -t ed25519 -C "your-name-ssl-manager" -f ~/.ssh/id_ed25519_ssl_manager
```

Set a passphrase when prompted.

```
# View the public key
cat ~/.ssh/id_ed25519.pub
cat ~/.ssh/id_ed25519_ssl_manager.pub   # named key

# Copy to clipboard (install xclip first if needed)
xclip -selection clipboard < ~/.ssh/id_ed25519.pub          # X11
wl-copy < ~/.ssh/id_ed25519.pub                             # Wayland
```

Connecting with a named key:

```
ssh -i ~/.ssh/id_ed25519_ssl_manager user@<server-ip>

# With tunnel
ssh -i ~/.ssh/id_ed25519_ssl_manager -L 5001:127.0.0.1:5001 user@<server-ip>
```

Or add to `~/.ssh/config`:

```
Host ssl-manager
    HostName <server-ip>
    User <username>
    IdentityFile ~/.ssh/id_ed25519_ssl_manager
    LocalForward 5001 127.0.0.1:5001
    ServerAliveInterval 60
```

Windows

Option A — OpenSSH (recommended, built into Windows 10/11)

Open **PowerShell** or **Windows Terminal** and check whether a key already exists:

```
Test-Path $env:USERPROFILE\.ssh\id_ed25519
```

A result of `True` means a key exists. You can reuse it, or generate a named key to keep SSL Manager separate.

If no key exists, or you want a dedicated key:

```
# Default name
ssh-keygen -t ed25519 -C "your-name-ssl-manager"

# Named key
ssh-keygen -t ed25519 -C "your-name-ssl-manager" -f
        "$env:USERPROFILE\.ssh\id_ed25519_ssl_manager"
```

Set a passphrase when prompted. Keys are written to `C:\Users\<you>\.ssh\`.

```
# View the public key
type $env:USERPROFILE\.ssh\id_ed25519.pub
type $env:USERPROFILE\.ssh\id_ed25519_ssl_manager.pub   # named key

# Copy the public key to the clipboard
Get-Content $env:USERPROFILE\.ssh\id_ed25519.pub | Set-Clipboard
```

Connecting with a named key:

```
ssh -i $env:USERPROFILE\.ssh\id_ed25519_ssl_manager user@<server-ip>
ssh -i $env:USERPROFILE\.ssh\id_ed25519_ssl_manager -L 5001:127.0.0.1:5001 user@<server-ip>
```

Or add to `C:\Users\<you>\.ssh\config`:

```
Host ssl-manager
    HostName <server-ip>
    User <username>
    IdentityFile C:\Users\<you>\.ssh\id_ed25519_ssl_manager
    LocalForward 5001 127.0.0.1:5001
    ServerAliveInterval 60
```

Option B — Git Bash

If you have Git for Windows installed, open **Git Bash** and follow the Linux instructions above. Keys are stored in `~/.ssh/` within Git Bash, which maps to `C:\Users\<you>\.ssh\`.

Option C — PuTTYgen (PuTTY users)

If you use PuTTY for SSH sessions:

1. Open **PuTTYgen** (included with PuTTY)
2. Check whether an existing `.ppk` file exists — if so, load and reuse it via **File → Load private key**
3. To create a new key: select **EdDSA** and click **Generate**, then move the mouse to generate entropy

4. Set a **Key passphrase**
5. Click **Save private key** — save the `.ppk` file (e.g. `ssl-manager.ppk`)
6. Copy the text in the **Public key for pasting into OpenSSH authorized_keys** box

To specify the key in a PuTTY session: go to **Connection** → **SSH** → **Auth** → **Credentials** and set the **Private key file** to your `.ppk`.

> The `.ppk` format is PuTTY-specific. When asked for your public key, provide the text from the "Public key for pasting into OpenSSH" box, not the `.ppk` file itself.

Find your current public IP

You will need your public IP address to populate `allowed_ssh_ips` in `terraform.tfvars`.

```
# macOS / Linux
curl -s https://checkip.amazonaws.com

# Windows (PowerShell)
(Invoke-WebRequest -Uri "https://checkip.amazonaws.com").Content.Trim()
```

AWS (EC2)

SSL Manager can be deployed to AWS using the Terraform configuration in `deploy/aws/`. It provisions a hardened EC2 instance (Ubuntu 24.04 LTS) with an encrypted EBS root volume and a security group that restricts SSH access to specified IP addresses only.

Full instructions: DEPLOY-AWS.md

Summary of steps

```
# 1. Install prerequisites — see "Cloud Deployment Prerequisites" above

# 2. Authenticate
aws configure
aws sts get-caller-identity     # confirm the correct account

# 3. Configure
cd deploy/aws
cp terraform.tfvars.example terraform.tfvars
# Edit terraform.tfvars: set admin_ssh_public_key and allowed_ssh_ips

# 4. Deploy
terraform init
terraform plan
terraform apply

# 5. Install SSL Manager on the provisioned instance
ssh ubuntu@<public_ip_from_output>
# Then follow the bare metal steps above from "Get the code"
```

What Terraform provisions

Resource	Detail
VPC, subnet, internet gateway	Isolated network with public routing
Security group	SSH (port 22) from `allowed_ssh_ips` only; all other inbound blocked
EC2 instance	`t3.small` (2 vCPU / 2 GB), Ubuntu 24.04 LTS
EBS root volume	30 GB gp3, encrypted with AWS-managed key
Elastic IP	Static public IP, persists through reboots

Adding SSH IPs later

```
# terraform.tfvars
allowed_ssh_ips = [
  "203.0.113.10/32",   # existing
  "198.51.100.5/32",   # new IP
]
```

```
terraform apply   # updates the security group in seconds, no restart needed
```

Azure (VM)

SSL Manager can be deployed to Azure using the Terraform configuration in `deploy/azure/`. It provisions a hardened Linux VM (Ubuntu 24.04 LTS) with hypervisor-level disk encryption and a Network Security Group that restricts SSH access to specified IP addresses only.

Full instructions: DEPLOY-AZURE.md

Summary of steps

```
# 1. Install prerequisites — see "Cloud Deployment Prerequisites" above

# 2. Authenticate
az login
az account show                    # confirm the correct subscription

# 3. Register the EncryptionAtHost feature (one-time per subscription)
az feature register --name EncryptionAtHost --namespace Microsoft.Compute
az feature show    --name EncryptionAtHost --namespace Microsoft.Compute \
  --query "properties.state" -o tsv
# Wait until output is "Registered" (5–10 minutes)
az provider register --namespace Microsoft.Compute

# 4. Configure
cd deploy/azure
cp terraform.tfvars.example terraform.tfvars
# Edit terraform.tfvars: set admin_ssh_public_key and allowed_ssh_ips

# 5. Deploy
terraform init
terraform plan
terraform apply

# 6. Install SSL Manager on the provisioned VM
ssh sslmgr@<public_ip_from_output>
# Then follow the bare metal steps above from "Get the code"
```

What Terraform provisions

Resource	Detail
Resource group, VNet, subnet	Isolated network
Network Security Group	SSH (port 22) from `allowed_ssh_ips` only; all other inbound blocked
VM	`Standard_B1ms` (1 vCPU / 2 GB), Ubuntu 24.04 LTS
OS disk	30 GB Premium SSD, encrypted at host

Resource	Detail
Static public IP	Standard SKU, persists through reboots

Adding SSH IPs later

```
# terraform.tfvars
allowed_ssh_ips = [
  "203.0.113.10/32",   # existing
  "198.51.100.5/32",   # new IP
]
```

```
terraform apply   # updates the NSG rule in seconds, no restart needed
```

Docker (Local Development)

Docker is provided for local development and testing only. It runs the Flask application directly without nginx, gunicorn, systemd, or the hardening applied by `install.sh`. **Do not use Docker in production.**

App only

```
docker compose up --build
```

The app is available at `http://localhost:5001`.

App + backup test service

Adds a `backup-test` container that runs `backup.sh` 10 seconds after startup, then every hour. Use this to verify backup execution and audit log entries without waiting for the systemd timer.

```
docker compose -f docker-compose.yml -f docker-compose.test.yml up --build
```

Watch backup output:

```
docker compose -f docker-compose.yml -f docker-compose.test.yml logs -f backup-test
```

App + design seed data

Populates the database with representative seed data (certificates, chains, CAs, users) for UI development and screenshot generation:

```
docker compose -f docker-compose.yml -f docker-compose.design.yml up --build
```

Seed credentials: `designer / design123` (superadmin), `alice / design123` (user). See `seed_design.py` for the full dataset.

Full developer workflow documentation, including the complete Docker command reference, is in WORKFLOW.md — Section 1.

Managing SSH Users on the Server

This section applies to all deployment methods. Perform these steps as the initial admin user after connecting to the server for the first time.

Creating a user account

```
# Create a standard user (no sudo)
sudo adduser alice

# Create a user with sudo access
sudo adduser bob
sudo usermod -aG sudo bob      # Ubuntu
sudo usermod -aG wheel bob     # RHEL-family
```

`adduser` / `useradd` is interactive — it prompts for a password. The password is used only for `sudo` prompts on the server itself; SSH login uses keys.

> **Tip:** For users who only need SSH tunnel access to the SSL Manager web UI and will never run commands on the server, a standard account without sudo is sufficient.

Adding a user's SSH public key

Each user generates their key pair on their own machine and sends their **public key** to the administrator.

As the admin, placing a key for another user:

```
sudo mkdir -p /home/alice/.ssh
sudo chmod 700 /home/alice/.ssh
echo "ssh-ed25519 AAAAC3Nz...rest-of-key... alice-laptop" \
  | sudo tee -a /home/alice/.ssh/authorized_keys
sudo chmod 600 /home/alice/.ssh/authorized_keys
sudo chown -R alice:alice /home/alice/.ssh
```

Adding multiple keys for one user

Each line in `authorized_keys` is one public key. A user can have keys for multiple devices:

```
sudo tee -a /home/alice/.ssh/authorized_keys <<'EOF'
ssh-ed25519 AAAAC3Nz...key-one... alice-laptop
ssh-ed25519 AAAAC3Nz...key-two... alice-home-desktop
EOF
sudo chmod 600 /home/alice/.ssh/authorized_keys
```

Disabling password authentication (recommended)

Once all users have working key-based login, disable password authentication:

```
sudo nano /etc/ssh/sshd_config
```

Set or confirm:

```
PasswordAuthentication no
PubkeyAuthentication yes
PermitRootLogin no
AuthorizedKeysFile .ssh/authorized_keys
```

```
sudo sshd -t          # test config before applying
sudo systemctl reload ssh
```

> **Important:** Do not close your current session until you have confirmed key-based login works in a new terminal window.

Revoking access

To remove a user's access, delete their line in `authorized_keys`:

```
sudo nano /home/alice/.ssh/authorized_keys
# Delete the line containing the key to revoke
```

To remove the user entirely:

```
sudo deluser --remove-home alice      # Ubuntu
sudo userdel --remove alice           # RHEL-family
```

Upgrading from Previous Versions

The `--upgrade` flag handles all routine upgrades: it backs up the database, then refreshes the application files, Python dependencies, systemd unit, and nginx config. Your `SECRET_KEY`, database, and port/worker settings are preserved. Re-running the installer with no flag auto-detects the existing install and does the same thing. **Schema changes are applied automatically** on the next service start — no manual SQL steps are ever required.

```
sudo bash install.sh --upgrade          # Ubuntu
sudo bash install-rhel.sh --upgrade     # RHEL-family
```

Version history and migration notes

Current release — Expiry notifications, installer & startup reliability

No manual steps required on upgrade.

New features: - **Expiry notification emails** — an optional daily digest (superadmin → Settings → Notifications) emails a colour-coded list of certificates, CAs, and intermediates expiring within a configurable threshold. Delivered by the new `ssl-manager-notify.timer`.

Reliability fixes: - **Worker-boot race fixed** — concurrent gunicorn workers no longer race on schema initialization at startup. Previously a boot could fail with `table ... already exists` (service `status=3`, "Worker failed to boot"); schema bootstrap is now serialized with an

exclusive lock. - **Installer auto-upgrade** — re-running `install.sh` / `install-rhel.sh` on an existing host now detects the installation and upgrades instead of starting a fresh interactive install. A new `--reinstall` flag forces the interactive installer. - **`SECRET_KEY` preserved on re-run** — the installer no longer regenerates the key on an existing install. Previously this could rotate the key and make stored SMTP/OAuth secrets undecryptable.

Schema migrations (automatic): - `notification_config` — new table created to store expiry-notification settings.

Smart bundle import

Added in a prior update. No manual steps required on upgrade.

New features: - **Smart P12/PFX import** — uploading a `.p12` or `.pfx` file now shows a live File Analysis preview (key type, certificate CN, expiry, detected intermediates, chain action) before import is confirmed. - **Keypair import** — a new Import Private Key + Certificate flow accepts a separate PEM key and certificate (or bundle, including P7B). Live preview confirms key/certificate match before import. - **Smart bundle upload** — uploading a signed certificate via the Certificate Detail page now performs role-based detection: the leaf certificate is identified by its CA flag, intermediates are split out and deduplicated against the assigned chain. P7B bundles are supported. - **Security update** — `cryptography` library updated to 46.0.7 (addresses CVE-2025-61727).

v1.2 — Password reset and SMTP

New features: - **Password reset by email** — users can request a password reset link from the login page. Requires SMTP to be configured (see Configure Email). - **Microsoft 365 and Google OAuth** — SMTP authentication via OAuth 2.0, removing the need to store an application password. - **Automatic session invalidation** — changing a user's password immediately invalidates all existing sessions for that account. - **15-minute idle timeout** — unauthenticated sessions are invalidated after 15 minutes of inactivity.

Schema migrations (automatic): - `user.session_version` — integer column added to support session invalidation on password change. - `smtp_config.*` — new table (and additional OAuth columns) created to store SMTP configuration.

Upgrade action required: None — migrations run automatically on first start after upgrade.

v1.1 — Certificate Authorities (internal CA)

New features: - **Internal CA module** — create self-signed root CAs entirely within SSL Manager and use them to sign pending certificates without an external CA portal. - **Certificate Profiles** — reusable templates for CSR subject fields (organisation, country, key size, etc.). - **Named Certificate Chains** — intermediate certificates are now grouped into named chains that can be shared across multiple certificates. - **Import CSR** — import an externally generated CSR (from a network appliance, load balancer, etc.) as a pending certificate record.

Schema migrations (automatic): - `cert_chain` — new table for named certificate chains. - `intermediate_cert.chain_id` — foreign key added to link intermediates to a chain. - `certificate.chain_id` — foreign key added to link certificates to a chain. - `certificate.profile_id` — foreign key added to link certificates to a profile. - `settings.name` / `settings.is_default` — columns added to support named profiles.

Upgrade action required: None — all migrations and data backfills run automatically on first start. Existing intermediates are migrated into a “Default Chain” automatically.

v1.0 — Initial release

Base feature set: certificate lifecycle management (generate key + CSR, upload signed cert), intermediate certificate management, user accounts, audit log, daily backup timer.

Choosing Between AWS and Azure

Consideration	AWS	Azure
Disk encryption setup	None — `encrypted = true` works immediately	One-time `az feature register` per subscription required
Cheapest option	t4g.small (ARM Graviton2) ~$12/month + ~$1.60 EBS	B1ms ~$15/month
Minimum viable	t4g.micro ~$6/month + EBS	B1s ~$8/month
Static IP billing	EIP free when attached; ~$0.005/hr when unattached	Included in VM cost
Existing account	Prefer if you have AWS credits or consolidated billing	Prefer if you have Azure credits or EA
Admin username	`ubuntu` (fixed by AMI)	Configurable (default: `sslmgr`)

See REQUIREMENTS.md for full hardware sizing rationale and a complete provider comparison table.

SSL Manager — AWS Deployment

Terraform configuration that provisions a hardened AWS EC2 instance running SSL Manager.

What is created

Resource	Details
VPC	Isolated network (`10.0.0.0/16`) with DNS enabled
Internet Gateway	Provides outbound internet access (for apt, pip, OS updates)
Subnet	Public subnet (`10.0.1.0/24`) in `<region>a`
Route Table	Default route → Internet Gateway
Security Group	SSH (port 22) allowed from `allowed_ssh_ips` only; all other inbound blocked by default
Key Pair	SSH public key registered in EC2
EC2 Instance	Ubuntu 24.04 LTS, `t3.small` (2 vCPU / 2 GB RAM)
Root EBS Volume	30 GB gp3, encrypted with AWS-managed key
Elastic IP	Static public IP; persists through stop/start and reboots

EBS encryption

`encrypted = true` on the root EBS volume encrypts all data at rest using the AWS-managed key (`alias/aws/ebs`) at no additional cost. This requires no prerequisite registration and is applied immediately at provisioning time, covering the SQLite database, log files, backup archives, SSL certificate private keys, and CA private keys generated by the internal CA module.

To use a Customer Managed Key (CMK) from AWS KMS instead, uncomment and set `kms_key_id` in the `root_block_device` block in `main.tf` .

Security Group IP management

All permitted SSH source addresses are stored in the `allowed_ssh_ips` Terraform variable. Each IP gets its own ingress rule (AWS best practice). To add or remove an IP:

1. Edit `terraform.tfvars` — add or remove entries from `allowed_ssh_ips`

2. Run `terraform apply`

The security group rules update in place in seconds; no instance restart is required.

Prerequisites

1. Tools

Tool	Install
Terraform	`brew install terraform` / hashicorp.com
AWS CLI v2	`brew install awscli` / AWS docs

2. AWS credentials

Configure credentials for the account and region you want to deploy into:

```
aws configure
# Prompts for: AWS Access Key ID, Secret Access Key, region, output format
```

Or for SSO / assumed roles:

```
aws sso login --profile your-profile
export AWS_PROFILE=your-profile
```

Confirm the correct account is active:

```
aws sts get-caller-identity
```

3. SSH key pair

If you do not already have an SSH key pair:

```
ssh-keygen -t ed25519 -C "ssl-manager-aws"
# Keys written to ~/.ssh/id_ed25519 and ~/.ssh/id_ed25519.pub
```

Deploy

1. Configure variables

```
cd deploy/aws
cp terraform.tfvars.example terraform.tfvars
```

Edit `terraform.tfvars` and set at minimum:

- `admin_ssh_public_key` — contents of `~/.ssh/id_ed25519.pub`
- `allowed_ssh_ips` — your current public IP (find it with `curl -s https://checkip.amazonaws.com`)
- `timezone` — tz database name for the server clock, e.g. `"America/New_York"` (controls backup filenames, log timestamps, and the daily backup timer; defaults to `"UTC"`)

2. Initialise and apply

```
terraform init
terraform plan    # review what will be created
terraform apply   # type 'yes' to confirm
```

`terraform apply` completes in approximately 1–3 minutes. On success, the outputs section displays:

```
Outputs:

allowed_ssh_ips   = ["203.0.113.10/32"]
ami_id            = "ami-0xxxxxxxxxxxxxxx"
ami_name          = "ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-20240423"
ebs_encrypted     = true
instance_id       = "i-0xxxxxxxxxxxxxxx"
public_ip_address = "54.x.x.x"
region            = "us-east-1"
ssh_command       = "ssh ubuntu@54.x.x.x"
ssh_tunnel_command = "ssh -L 5001:127.0.0.1:5001 ubuntu@54.x.x.x"
```

3. Connect and install SSL Manager

```
# Connect to the instance
ssh ubuntu@<public_ip_address>

# On the instance — update packages
sudo apt-get update && sudo apt-get upgrade -y

# Clone the repository (or copy files via scp)
git clone https://github.com/your-org/ssl-manager.git
cd ssl-manager

# Run the installer
sudo bash install.sh
```

Follow the interactive installer prompts (port, Gunicorn workers, secret key). When complete, the SSL Manager service is running and the daily backup timer is enabled.

4. Open the web UI via SSH tunnel

From your **local machine**:

```
ssh -L 5001:127.0.0.1:5001 ubuntu@<public_ip_address>
```

Then open `http://localhost:5001` in your browser and complete first-run setup.

For a persistent tunnel configuration, add to `~/.ssh/config`:

```
Host ssl-manager
    HostName <public_ip_address>
    User ubuntu
    IdentityFile ~/.ssh/id_ed25519
    LocalForward 5001 127.0.0.1:5001
    ServerAliveInterval 60
```

Then simply `ssh ssl-manager`.

Using a Graviton (ARM) instance

Graviton2 instances (`t4g`) offer approximately 20% lower cost for equivalent performance. To switch:

1. In `terraform.tfvars`, set:

```
instance_type = "t4g.small"
ami_arch      = "arm64"
```

2. Run `terraform apply`

The AMI data source selects the correct Ubuntu 24.04 ARM image automatically. All SSL Manager dependencies support ARM — the `cryptography` package includes ARM native extensions.

> **Note:** Switching between `x86_64` and `arm64` replaces the EC2 instance. Take a manual backup before switching architecture.

Adding SSH IPs

Edit `terraform.tfvars`:

```
allowed_ssh_ips = [
  "203.0.113.10/32",   # Office
  "198.51.100.5/32",   # New IP to add
]
```

Apply:

```
terraform apply
```

The security group rules update in seconds. No instance restart needed.

Verify EBS encryption

After the instance is running:

```
aws ec2 describe-volumes \
  --filters "Name=attachment.instance-id,Values=$(terraform output -raw instance_id)" \
  --query "Volumes[*].{ID:VolumeId,Encrypted:Encrypted,KmsKeyId:KmsKeyId}" \
  --output table
# Encrypted column should show True
```

> **Note:** AWS EBS encryption is applied at the hypervisor/storage layer and is transparent to the guest OS. Running `lsblk` inside the instance will not show any encryption indicator — the volume appears as a normal block device. The AWS CLI command above is the authoritative verification method.

Upgrade SSL Manager

After pulling new code, copy it to the instance and run the upgrade:

```
# From your local machine — copy updated files
scp -r wsgi.py app/ backup.sh requirements.txt \
    ubuntu@<public_ip>:/tmp/ssl-manager-update/

# On the instance
sudo cp -r /tmp/ssl-manager-update/* /opt/ssl-manager/
sudo bash /opt/ssl-manager/install.sh --upgrade
```

> The upgrade preserves your `SECRET_KEY`, database, and port/worker settings while refreshing the application files, systemd unit, and nginx config. Re-running `install.sh` with no flag does the same thing — it auto-detects the existing install and upgrades.

Network considerations

Outbound SMTP

The default Terraform security group allows **all outbound traffic**, so no additional rule is needed for email delivery. However, some AWS accounts or VPCs have outbound restrictions. If password reset emails are not delivered, verify that outbound TCP on port 587 (STARTTLS) or 465 (SSL) is not blocked by a VPC Network ACL or an account-level firewall policy.

To check the effective outbound rules on the instance's security group:

```
aws ec2 describe-security-groups \
  --group-ids $(terraform output -raw security_group_id) \
  --query "SecurityGroups[*].IpPermissionsEgress" \
  --output table
```

Tear down

```
terraform destroy
```

This terminates the EC2 instance and releases the Elastic IP. The EBS volume is deleted (`delete_on_termination = true`). Take a manual backup first:

```
ssh ubuntu@<public_ip> \
  "sudo bash /opt/ssl-manager/backup.sh --dest /tmp/ssl-final-backup && \
   sudo ls -lh /tmp/ssl-final-backup/"

# Copy the backup archive locally before destroying
scp "ubuntu@<public_ip>:/tmp/ssl-final-backup/*.db.gz" .
```

> **Elastic IP note:** An unassociated Elastic IP is billed at ~$0.005/hour. `terraform destroy` releases it, stopping the charge.

SSL Manager — Azure Deployment

Terraform configuration that provisions a hardened Azure VM running SSL Manager.

What is created

Resource	Details
Resource Group	Contains all SSL Manager resources
Virtual Network + Subnet	Isolated network (`10.0.0.0/16` / `10.0.1.0/24`)
Network Security Group	SSH (port 22) allowed from `allowed_ssh_ips` only; all other inbound blocked
Static Public IP	Standard SKU — address persists through VM restarts
Network Interface	NSG applied at both NIC and subnet level (defence in depth)
Linux VM	Ubuntu 24.04 LTS, `Standard_B1ms` (1 vCPU / 2 GB RAM)
OS Disk	30 GB Premium SSD, encrypted at host

Disk encryption

`encryption_at_host_enabled = true` encrypts all data written by the VM at the hypervisor level — OS disk, temporary disk, and their caches — using Azure platform-managed keys. This requires a one-time subscription feature registration (see Prerequisites below).

This approach covers everything written to disk by the SSL Manager application: the SQLite database, log files, backup archives, SSL certificate private keys, and CA private keys generated by the internal CA module.

NSG IP management

All permitted SSH source addresses are stored in the `allowed_ssh_ips` Terraform variable. To add or remove an IP:

1. Edit `terraform.tfvars` — add or remove entries from `allowed_ssh_ips`
2. Run `terraform apply`

The NSG rule updates in place in seconds; no VM restart is required.

Prerequisites

1. Tools

Tool	Install
Terraform	`brew install terraform` / hashicorp.com
Azure CLI	`brew install azure-cli` / Microsoft docs

2. Azure login

```
az login
az account show   # confirm the correct subscription is active
```

To switch subscriptions:

```
az account set --subscription "<subscription-id-or-name>"
```

3. Register the EncryptionAtHost feature

This is a one-time step per subscription. It takes 5–10 minutes.

```
az feature register --name EncryptionAtHost --namespace Microsoft.Compute

# Poll until state is "Registered"
az feature show --name EncryptionAtHost --namespace Microsoft.Compute \
  --query "properties.state" -o tsv

az provider register --namespace Microsoft.Compute
```

4. SSH key pair

If you do not already have an SSH key pair:

```
ssh-keygen -t ed25519 -C "ssl-manager-azure"
# Keys written to ~/.ssh/id_ed25519 and ~/.ssh/id_ed25519.pub
```

Deploy

1. Configure variables

```
cd deploy/azure
cp terraform.tfvars.example terraform.tfvars
```

Edit `terraform.tfvars` and set at minimum:

- `admin_ssh_public_key` — contents of `~/.ssh/id_ed25519.pub`
- `allowed_ssh_ips` — your current public IP (find it with `curl -s https://checkip.amazonaws.com`)
- `timezone` — tz database name for the server clock, e.g. `"America/New_York"` (controls backup filenames, log timestamps, and the daily backup timer; defaults to `"UTC"`)

2. Initialise and apply

```
terraform init
terraform plan    # review what will be created
terraform apply   # type 'yes' to confirm
```

`terraform apply` completes in approximately 2–4 minutes. On success, the outputs section displays:

```
Outputs:

allowed_ssh_ips            = ["203.0.113.10/32"]
encryption_at_host_enabled = true
public_ip_address          = "20.x.x.x"
resource_group_name        = "ssl-manager-rg"
ssh_command                = "ssh sslmgr@20.x.x.x"
ssh_tunnel_command         = "ssh -L 5001:127.0.0.1:5001 sslmgr@20.x.x.x"
vm_name                    = "ssl-manager-vm"
```

3. Connect and install SSL Manager

```
# Connect to the VM
ssh sslmgr@<public_ip_address>

# On the VM — update packages
sudo apt-get update && sudo apt-get upgrade -y

# Clone the repository (or copy files via scp)
git clone https://github.com/your-org/ssl-manager.git
cd ssl-manager

# Run the installer
sudo bash install.sh
```

Follow the interactive installer prompts (port, Gunicorn workers, secret key). When complete, the SSL Manager service is running and the daily backup timer is enabled.

4. Open the web UI via SSH tunnel

From your **local machine**:

```
ssh -L 5001:127.0.0.1:5001 sslmgr@<public_ip_address>
```

Then open `http://localhost:5001` in your browser and complete first-run setup.

For persistent tunnel configuration, add to `~/.ssh/config`:

```
Host ssl-manager
    HostName <public_ip_address>
    User sslmgr
    IdentityFile ~/.ssh/id_ed25519
    LocalForward 5001 127.0.0.1:5001
    ServerAliveInterval 60
```

Then simply `ssh ssl-manager`.

Adding SSH IPs

Edit `terraform.tfvars`:

```
allowed_ssh_ips = [
  "203.0.113.10/32",   # Office
  "198.51.100.5/32",   # New IP to add
]
```

Apply:

```
terraform apply
```

The NSG rule updates in seconds. No VM restart needed.

Verify disk encryption

After the VM is running:

```
az vm show \
  --resource-group ssl-manager-rg \
  --name ssl-manager-vm \
  --query "securityProfile.encryptionAtHost" \
  --output tsv
# Expected output: true
```

Upgrade SSL Manager

After pulling new code, upload it to the VM and run the upgrade:

```
# From your local machine — copy updated files
scp -r wsgi.py app/ backup.sh requirements.txt \
    sslmgr@<public_ip>:/tmp/ssl-manager-update/

# On the VM
sudo cp -r /tmp/ssl-manager-update/* /opt/ssl-manager/
sudo bash /opt/ssl-manager/install.sh --upgrade
```

> The upgrade preserves your `SECRET_KEY`, database, and port/worker settings while refreshing the application files, systemd unit, and nginx config. Re-running `install.sh` with no flag does the same thing — it auto-detects the existing install and upgrades.

Network considerations

Outbound SMTP

The Terraform NSG allows **all outbound traffic** by default, so no additional rule is needed for email delivery. However, some Azure subscriptions or policies restrict outbound traffic. If password reset emails are not delivered, verify that outbound TCP on port 587 (STARTTLS) or 465 (SSL) is not blocked by a subscription-level Azure Policy or a custom NSG rule.

To list effective outbound security rules for the VM's NIC:

```
az network nic list-effective-nsg \
  --resource-group ssl-manager-rg \
  --name $(az network nic list --resource-group ssl-manager-rg \
           --query "[0].name" -o tsv) \
  --query "effectiveSecurityRules[?direction=='Outbound']" \
  --output table
```

Tear down

```
terraform destroy
```

This removes all resources in the resource group. The SQLite database is on the VM's OS disk and will be deleted. Take a manual backup first:

```
ssh sslmgr@<public_ip> \
  "sudo bash /opt/ssl-manager/backup.sh --dest /tmp/ssl-final-backup && \
   sudo ls -lh /tmp/ssl-final-backup/"

# Copy the backup archive locally before destroying
scp "sslmgr@<public_ip>:/tmp/ssl-final-backup/*.db.gz" .
```